SYSTÈMES D’EXPLOITATION AVANCÉS

RAPPORT DE PROJET

Jeu de bataille navale multi-joueurs

Application console en C11 et threads POSIX

Conception • Synchronisation • Vérification

Établissement :	ESPRIM
Étudiant :	Yazid Slim
Année universitaire :	2025–2026

Rapport technique fondé sur le code source, le README et les résultats de tests consignés dans le projet.

1. Introduction

Le projet consiste à réaliser une bataille navale multijoueurs utilisable dans un terminal sous Ubuntu Linux. Le sujet du module « Systèmes d’exploitation avancés » met l’accent sur la création d’un thread par joueur et sur la synchronisation des accès aux grilles. Le programme développé répond à ce cadre au moyen de C11, de l’interface POSIX threads et d’une interface textuelle. Il n’exige aucune bibliothèque applicative externe.

Deux à quatre joueurs participent à une partie à tours de rôle. Chacun dispose de sa propre grille de 10 × 10 cases et d’une flotte placée automatiquement. Le joueur actif choisit un adversaire et une coordonnée ; le programme valide le tir, met à jour la grille concernée et annonce le résultat. La partie s’achève lorsqu’un seul joueur possède encore au moins un navire à flot.

2. Objectifs et périmètre

Les objectifs techniques sont de représenter explicitement les joueurs, les navires et les grilles ; de confier des actions effectives à un thread par joueur ; d’empêcher les accès concurrents incohérents ; et de garantir une fin de partie ainsi qu’un arrêt propre des threads. Le programme doit aussi rendre visibles les tirs réussis et ratés sans dévoiler les navires adverses encore intacts.

3. Architecture logicielle

Le fichier game.h définit les structures communes. Board contient la position des navires, l’historique des tirs reçus, les cinq Ship et le nombre de segments restant à flot. Player réunit une grille, un identifiant de joueur, un pthread_t, son état vivant, une variable de condition et une boîte de commande. Game centralise les joueurs, le tour courant, le vainqueur, le résultat de la dernière action et les primitives de synchronisation.

Le moteur engine.c initialise les flottes, valide les tirs et calcule leurs conséquences, indépendamment de la console. Le module workers.c crée les threads et organise l’échange des commandes. Le module ui.c lit les lignes saisies et affiche les grilles. Enfin, main.c traite les arguments, démarre les workers, pilote la partie et assure leur arrêt. Cette séparation facilite la lecture et permet de tester les règles du jeu sans interaction humaine.

4. Threads POSIX et synchronisation

Le programme crée un worker POSIX par joueur avec pthread_create. Le thread principal est le seul à lire stdin et à écrire les messages. Cette répartition est nécessaire, car plusieurs appels concurrents à la lecture du même terminal produiraient des commandes ambiguës. Les workers n’attendent pas activement : chacun dort sur sa variable de condition pthread_cond_t jusqu’à l’arrivée d’une commande ou d’une demande d’arrêt.

Lorsqu’un joueur tire, le principal dépose une commande dans sa boîte et le réveille par pthread_cond_signal. Le worker attaquant vérifie la cible, les coordonnées, le tour en cours et l’absence de tir répété. Si le tir est valable, il place une commande de tir entrant dans la boîte du défenseur. Le worker défenseur modifie sa propre grille, calcule le résultat, met éventuellement à jour l’élimination et le vainqueur, puis signale le principal. Ainsi, les deux rôles de thread participent effectivement à l’action de jeu.

4.1 Protection de l’état partagé

Un seul pthread_mutex_t protège les grilles, les boîtes de commandes, le tour, les états de vie, le résultat et le drapeau d’arrêt. Le tir et ses effets sont calculés pendant la même section critique : un coup ne peut donc pas être enregistré deux fois ni laisser une victoire partiellement mise à jour. Les attentes utilisent pthread_cond_wait dans une boucle qui réévalue leur condition après chaque réveil, y compris en cas de réveil intempestif.

Une seule commande de jeu est en cours à la fois. Le principal attend son résultat avant de demander un nouveau tir ; aucune boîte ne doit être écrasée. Le verrou est libéré pendant l’attente des commandes et aucun appel de lecture de stdin n’est effectué sous mutex. Le choix d’un verrou unique rend le protocole simple à justifier et évite les inversions d’ordre de verrouillage.

4.2 Arrêt et libération des ressources

Lorsque le joueur termine la saisie par EOF, ou lorsque la partie se termine, le principal active le drapeau d’arrêt, réveille tous les workers, puis les attend avec pthread_join. Il détruit ensuite les variables de condition et le mutex. Les chemins d’échec pendant la création des primitives ou des threads nettoient les ressources déjà créées. Les grilles et structures du jeu sont stockées dans Game ; le programme ne réserve pas ces données par allocation dynamique.

5. Fonctionnement du jeu

Chaque flotte comprend cinq navires de longueurs 5, 4, 3, 3 et 2. Le moteur tente des positions horizontales ou verticales, refuse toute sortie de grille ou superposition, puis inscrit les segments. Un générateur pseudo-aléatoire local reçoit une graine ; une valeur fournie sur la ligne de commande rend le placement reproductible pour les tests. La mise en place se déroule avant la création des threads.

Le joueur actif saisit trois nombres : le numéro de la cible, la ligne et la colonne, tous numérotés à partir de 1. Un tir sur soi-même, hors grille, vers un joueur éliminé ou vers une case déjà visée est refusé sans consommer le tour. Un tir valide annonce « raté », « touché » ou « bateau coulé ». Lorsque le dernier segment d’une flotte est atteint, ce joueur est éliminé et les tours futurs l’ignorent. La victoire est déclarée lorsque le compteur de joueurs vivants atteint un.

L’interface affiche la grille du joueur actif avec ses navires et les impacts reçus. Pour les adversaires, elle ne montre que les cases déjà visées : les navires non touchés restent cachés. Le mode --demo choisit volontairement des coordonnées à partir des grilles cachées afin de terminer rapidement une partie automatique ; ce mode est séparé du jeu normal et ne représente pas une stratégie de joueur humain.

6. Vérification et résultats

La commande make test a été exécutée avec succès. Elle compile l’application et lance les tests du moteur ainsi que les scénarios d’intégration. Les tests du moteur vérifient notamment 200 graines de placement à quatre joueurs, le nombre de segments de chaque navire, les tirs valides ou invalides, les tirs répétés, les navires coulés, l’élimination et la victoire. Les scénarios d’intégration couvrent une partie automatique, les entrées invalides, les tirs répétés et l’arrêt sur EOF. Les sorties observées sont « Engine tests passed. » et « Integration tests passed. »

Une vérification complémentaire a mené à terme 300 parties automatiques : 100 graines pour chacun des modes à deux, trois et quatre joueurs. Une exécution à quatre joueurs a montré cinq threads actifs, soit le principal et quatre workers. Les démonstrations demandées à deux et quatre joueurs ont toutes deux annoncé un vainqueur. La compilation avec -Wall, -Wextra et -Wpedantic n’a produit aucun avertissement.

Les démonstrations à deux et quatre joueurs ont également été exécutées avec AddressSanitizer et UndefinedBehaviorSanitizer sans anomalie signalée. La détection des fuites était désactivée car LeakSanitizer ne fonctionnait pas dans l’environnement de vérification ; ce point ne constitue donc pas une preuve d’absence de fuite.

7. Difficultés techniques et limites

La principale difficulté a été de concilier des threads réellement actifs avec une entrée console unique. Le protocole de boîtes de commandes permet au principal de garder la maîtrise du terminal tandis que chaque worker valide ses attaques et traite les coups reçus. Les variables de condition évitent toute boucle d’attente active. Le mutex unique rend les mises à jour cohérentes, tout en sérialisant les transitions de jeu, ce qui convient au choix de tours successifs.

La console étant physiquement partagée, l’application ne peut garantir qu’un autre joueur ne regarde pas la grille personnelle affichée pendant le tour courant. L’interface masque les navires adverses non touchés dans ses propres vues, mais la confidentialité entre personnes présentes devant le même écran reste une limite pratique. Par ailleurs, le mode de démonstration utilise intentionnellement les positions cachées ; il doit seulement servir à vérifier le déroulement complet.

8. Conclusion

Le projet aboutit à un jeu de bataille navale multijoueurs en console qui emploie un thread POSIX par joueur pour des tâches effectives, protège les grilles avec un mutex et termine les threads proprement. Les règles essentielles, la gestion des erreurs de saisie et la victoire ont été exercées par les tests exécutés. Le README décrit les commandes de compilation et d’utilisation, tandis que ce rapport documente les choix de conception et leurs limites.

Pour une remise universitaire, le code C, les en-têtes, le README et le présent rapport forment un ensemble cohérent.